

NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD



Parallel and Distributed Computing (LAB)

Lab Report: 02

Submitted to
Sir Farhad Riaz

Submitted By
Junaid Asif
(BSAI-144)

Submission Date: October 20, 2025

Task 1

Problem Statement

Write a program to demonstrate **loop-level parallelism** using `#pragma omp parallel` for.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
    int N = 20;
    #pragma omp parallel for
        for (int i = 0; i < N; i++){
            printf("Iteration %d is executed by thread %d\n", i,
omp_get_thread_num());
        }
    return 0;
}
```

Output

(Add screenshot here)

Explanation

- The directive `#pragma omp parallel for` divides loop iterations among multiple threads automatically.
- Each thread executes a **subset of the iterations**.
- **Why used:**
 - To achieve **data parallelism** — where multiple data elements (loop iterations) are processed in parallel.
- **Effect:**
 - Reduces total execution time for large loops.
 - Output order is non-deterministic because threads finish iterations asynchronously.

Task 2

Problem Statement

Modify Task 1 to manually control the number of threads (10) used for loop execution.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
    int N = 20;
    #pragma omp parallel for num_threads (10)
        for (int i=0; i < N; i++){
            printf("Iteration %d is executed by thread %d\n", i,
omp_get_thread_num());
        }
    return 0;
}
```

Output

(Add screenshot here)

Explanation

- The clause `num_threads(10)` specifies that **10 threads** will share the work of the loop.
- **Why used:**
 - To experiment with **manual thread allocation** and see its impact on scheduling.
- **Effect:**
 - Allows control over how many CPU cores are used.
 - For small loops, overhead may outweigh benefits; for larger loops, performance can improve.
 - Demonstrates **tunable parallelism** in OpenMP.

Task 3

Problem Statement

Write a program to calculate the **sum of first n numbers** using `#pragma omp parallel for`.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
    int sum = 0;
```

```
int n = 10;
#pragma omp parallel for
    for (int i = 0; i < n; i++){
        sum += i;
    }
printf("Sum of first %d numbers is %d\n", n, sum);
return 0;
}
```

Output

(Add screenshot here)

Explanation

- A parallel loop adds numbers to a shared variable sum.
- **Why used:**
 - To demonstrate **reduction-type problems** in parallel programming.
- **Effect:**
 - This version introduces a **data race** because multiple threads access sum simultaneously.
 - The result is **unreliable** without synchronization (e.g., reduction clause or critical section).
 - Highlights the importance of **thread-safe shared variable handling**.

Task 4

Problem Statement

Compute **partial sums** per thread and a final **global sum**, demonstrating the difference between **local** and **global** computations.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
    int n=20;
    #pragma omp parallel
    {
        int l_sum = 0;
        int thread_id = omp_get_thread_num();
        #pragma omp for
        for (int i = 0; i <= n; i++){
```

```
        l_sum += i;
    }
    printf("Thread %d local sum: %d\n", thread_id, l_sum);
}
int g_sum = 0;
for (int i=0; i <= n; i++)
{
    g_sum += i;
}
printf("Final sum of first %d numbers: %d\n", n, g_sum);
return 0;
}
```

Output

(Add screenshot here)

Explanation

- Each thread calculates its **local sum** independently.
- After the parallel region, a **global sum** is computed sequentially for comparison.
- **Why used:**
 - To show **how private variables prevent race conditions** and how data can be combined post parallel execution.
- **Effect:**
 - Demonstrates the use of **private workspace** per thread.
 - Reduces data races and helps in understanding **partial reduction** mechanisms.

Task 5

Problem Statement

Demonstrate **task parallelism** using the `#pragma omp sections` directive.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>

int main () {
    #pragma omp parallel
    {
```

```
#pragma omp sections
{
    #pragma omp section
    {
        printf("Task 1 executed by thread %d\n", omp_get_thread_num());
    }
    #pragma omp section
    {
        printf("Task 2 executed by thread %d\n", omp_get_thread_num());
    }
    #pragma omp section
    {
        printf("Task 3 executed by thread %d\n", omp_get_thread_num());
    }
}
return 0;
}
```

Output

(Add screenshot here)

Explanation

- The sections directive allows **different tasks** to be executed **in parallel**.
- Each section represents a separate block of code assigned to a different thread.
- **Why used:**
 - To implement **task parallelism**, where independent functions or tasks are run simultaneously.
- **Effect:**
 - Each section runs concurrently.
 - Efficient when tasks are **independent and unequal in workload**.

Task 6

Problem Statement

Perform **sum and product calculations** concurrently using OpenMP sections.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
```

```
int main () {
    int sum = 0;
    int product = 1;
    #pragma omp parallel
    {
        #pragma omp sections
        {
            #pragma omp section
            {
                for (int i=1;i<=5;i++)
                {
                    sum += i;
                }
                printf("The sum is %d and thread id is %d\n", sum,
omp_get_thread_num());
            }
            #pragma omp section
            {
                for (int i=1;i<=5;i++)
                {
                    product *= i;
                }
                printf("The product is %d and thread id is %d\n", product,
omp_get_thread_num());
            }
        }
    }
    return 0;
}
```

Output

(Add screenshot here)

Explanation

- Two different tasks (sum and product) are executed in **parallel sections**.
- **Why used:**
 - To show **simultaneous computation of independent operations**.
- **Effect:**
 - Demonstrates **task decomposition**.
 - The two computations don't interfere since they use separate variables.
 - Boosts performance when tasks are independent and CPU cores are free.

Task 7

Problem Statement / Question

Find **minimum and maximum values** in an array using omp sections.

Code

```
#include <stdio.h>
#include <omp.h>

int main() {
    int array[] = {56, 12, 3, 44, 5};
    int min = array[0];
    int max = array[0];

    #pragma omp parallel
    {
        #pragma omp sections
        {
            #pragma omp section
            {
                for (int i = 0; i < 5; i++) {
                    if (array[i] < min) {
                        min = array[i];
                    }
                }
                printf("The minimum element is %d\n", min);
            }

            #pragma omp section
            {
                for (int i = 0; i < 5; i++) {
                    if (array[i] > max) {
                        max = array[i];
                    }
                }
                printf("The maximum element is %d\n", max);
            }
        }
    }

    return 0;
```


}

Output*(Add screenshot here)***Explanation**

- Two **independent sections** execute simultaneously: one finds the minimum, the other finds the maximum.
- **Why used:**
 - To demonstrate **real-world task parallelism** — independent computations done in parallel.
- **Effect:**
 - Reduces total computation time compared to sequential execution.
 - Highlights efficient **division of work** among threads.

Comparative Analysis (Across All Tasks)

Aspect	Task 1–2	Task 3–4	Task 5–7	Why Used / Effect
Parallel Type	Data parallelism (parallel for)	Data parallelism with shared vs. private handling	Task parallelism (sections)	Shows both loop-level and functional parallelism in OpenMP.
Directive Used	#pragma omp parallel for	#pragma omp parallel + #pragma omp for	#pragma omp parallel + #pragma omp sections	Each directive defines how work is split — by iterations or by tasks.
Thread Control	Auto / Manual	Manual	Auto	Demonstrates OpenMP's flexibility in assigning threads.
Data Handling	Shared variables	Shared + Local	Independent per task	Introduces race conditions and their prevention.
Purpose	Loop iteration distribution	Summation with thread-specific computation	Multiple tasks executing concurrently	Covers all major parallelization models in OpenMP.
Performance Effect	Reduced loop time	Optimized but sensitive to	Improved throughput for	Balances CPU load between threads

Aspect	Task 1–2	Task 3–4	Task 5–7	Why Used / Effect
		shared data	multiple tasks	efficiently.
Why Used	For splitting iterations across cores	To test race handling and local variables	For running different tasks simultaneously	Explains OpenMP's versatility and control.
Output Order	Non-deterministic	Non-deterministic	Non-deterministic	Parallel threads execute asynchronously.
Common Issue	None	Data races (sum variable)	None (independent data)	Teaches synchronization needs in shared data cases.

Common Learnings Across All Tasks

- **OpenMP directives** greatly simplify parallelization of loops and tasks.
- **parallel for** = divides loop iterations among threads (data parallelism).
- **sections** = assigns distinct tasks to different threads (task parallelism).
- **Shared variables** cause race conditions unless handled with **private or reduction** clauses.
- The **number of threads** affects both speed and CPU utilization.
- Output from threads is **unordered**, reflecting real concurrent execution.

Key Takeaways

1. **Parallel For:** Automates splitting of iterative tasks.
2. **Sections:** Allows independent blocks of code to run simultaneously.
3. **Race Conditions:** Appear when shared variables are modified concurrently.
4. **Thread Control:** Enables performance tuning per hardware setup.
5. **Real-World Mapping:** Techniques apply to scientific computing, simulations, and AI workloads.

Conclusion

Lab 2 deepened understanding of **OpenMP's work-sharing constructs** — parallel for and sections.

We learned how to:

- Distribute loop iterations among threads (data parallelism).
- Execute independent functions concurrently (task parallelism).
- Manage variables safely across threads.

This lab bridges the gap between **basic thread creation** (Lab 1) and **efficient parallel task management**, a crucial skill for building high-performance and distributed systems.

